

cs-mail Protocol Specification

Relationship Requests, Permission, and Financial Obligations

C-SQD

Draft 0.9 — September 2026

Abstract

The *cs-mail* protocol makes permission to communicate a directed relationship. A sender who lacks permission may submit a relationship request with an initial encrypted message and one conditional charge. The recipient may accept, reject, or leave the request to expire. These outcomes determine refunds and forfeitures; permitted follow-up messages create no further charge. This specification defines the request lifecycle, authenticated decisions, financial obligations, and privacy boundaries. The C-SQD deployment adds a company-owned pool and annual equal member allocations under its financial-program profile.

1 Reading the Protocol Through One Request

Alice wants to establish permission to write to Bob. Bob has not accepted her public identity, and no scoped capability authorizes her message. Alice obtains terms for a dollar-denominated relationship request: processing component $C = 0.50$ and refundable collateral component $S = 5.00$. She authorizes a charge of 5.50 associated with that request. Submission occurs once funding is finalized, encrypted content is durable, and the service can commit a retryable delivery intent.

Bob sees one relationship request. If his settings permit follow-ups, Alice may add messages to it without another charge or a later decision deadline. Accepting Alice grants future permission and records a 5.50 refund obligation. Rejecting her retains 0.50 as processing revenue and records 5.00 as pending forfeiture. Bob does not acquire a personal claim on that collateral. If the request expires without a timely decision, the processing amount is retained and 5.00 is due back to Alice.

Those are protocol outcomes. The external provider may confirm a refund later. Likewise, a forfeiture does not become immediately available for distribution: the C-SQD profile separates unresolved exposure, eligible forfeitures, the corporate allocation, restricted member funds, and finalized member payables. This separation explains the objects and transitions that follow.

2 Status, Scope, and Conformance

This specification defines the request and permission contract. The Rust reference architecture identifies its implementation boundaries; the desktop plan describes the remaining product work. Conformance to the contract does not by itself establish readiness for live financial operations. The words **MUST**, **MUST NOT**, **SHOULD**, **SHOULD NOT**, and **MAY** describe conformance to this draft.

The protocol governs directed permission, one charge per request, request outcomes, financial obligations, authenticated admission, replay, and privacy. The section titled *C-SQD Financial Program* in the *Deployment and Migration Profile* is normative for a deployment advertising that

profile. It defines the company-owned pool and equal annual distribution rules. Parameters explicitly marked proposed there are not launch commitments. A live deployment **MUST** publish a complete, versioned policy before accepting real charges.

This document does not choose a banking partner, merchant arrangement, wire encoding, cryptographic suite, or jurisdiction-specific legal classification. The charge/refund structure and outsourcing of payments do not by themselves establish a regulatory exemption.

An implementation **MUST** identify the protocol and financial-policy versions used for every request. Old quotes and obligations **MUST NOT** be silently reinterpreted under this draft. Migration requires explicit treatment of existing requests, refunds, and entitlements; a changed document is not a migration event.

3 Permission, Requests, and Messages

3.1 The identities involved

A public protocol identity I_i is the addressable persona another participant can accept or block. Its controlling provider maintains authorized operational keys, status, and version. A provider-local private principal P_i attaches control, recovery, and anti-abuse continuity to one or more public identities. The general protocol can represent people or organizational principals; that scope does not establish organizational accounts in the current C-SQD product. C-SQD currently binds one identifiable person to one account, with multiple email addresses permitted and identity tied to a verified bank account. A principal is not automatically an enrolled distribution-program member. The [C-SQD domain model](#) and financial deployment profile define these product rules; distinct scoped identifiers retain their privacy roles.

Permission is directed from I_i to I_j . Accepting one identity **MUST NOT** accept another identity controlled by the same principal. Conversely, rotating identities, keys, devices, or providers **MUST NOT** erase principal-scoped request history for the same recipient. Outside its identity authority, the principal **MUST** be represented by purpose-limited scoped assertions, never a globally comparable identifier.

3.2 The enduring relationship

The relationship has state

$$A_{ij} \in \{\text{unknown}, \text{accepted}, \text{rejected}, \text{revoked}, \text{blocked}\}.$$

Its record contains a scoped directed-pair handle, monotonically increasing version, latest canonical event position, and transition time. Accepted ordinary communication requires no request charge. A valid scoped capability can authorize narrower communication without establishing acceptance. Blocking overrides both.

Unknown, rejected, and revoked relationships may be eligible for a new request under recipient and anti-abuse policy. Thus a request is needed when standing permission is absent, not only when no relationship record exists. Rejection declines a request; blocking prohibits further ordinary contact until removed.

3.3 The relationship request

A request is the economic unit. It **MUST** have a unique scoped identifier, directed relationship and generation, immutable quote, initial-message binding, original charge reference, creation time, submission deadline, optional submission time and decision deadline, state, and version. The request lifecycle has two nonterminal states. Submission commits the initial communication for delivery; message admission is the more general check of whether a particular message may proceed under a request, relationship, or capability. The states are:

PreparingSubmission

The request exists, but its initial communication has not yet been committed for delivery to the recipient. Funding and content preparation must finish before the submission deadline.

AwaitingRecipientDecision

Submission has committed and fixed the decision deadline. The recipient may decide, and permitted follow-ups may join the request.

The terminal outcomes are **Accepted**, **Rejected**, **Expired**, and **Cancelled**.

At most one nonterminal request **MUST** exist per directed public-identity pair. Creation and external-charge dispatch **MUST** enforce that constraint even across devices or different idempotency keys. A second submission returns the existing request or a stable conflict; it does not authorize another charge.

The charge purchases submission of a relationship request with an initial communication. It does not guarantee acceptance, attention, inbox prominence, or a response. Ordinary delivery recovery after submission does not undo the request. A sender cannot cancel a submitted request to avoid its outcome.

3.4 Messages while awaiting the recipient's decision

Submission creates one logical solicitation. Later messages may join only when authenticated recipient follow-up policy permits them. Each message retains its own identifier, ciphertext, envelope validity, and delivery state. It does not acquire a new financial decision window.

An allowed follow-up **MUST NOT** create another charge, solicitation generation, forfeiture or new-request history increment, and **MUST NOT** extend the request deadline. An exact replay returns the original admission result without consuming allowance twice. Recipient policy may disable follow-ups or bound count, rate, size, and declared scope. Its version and the allowance consumed **MUST** be recorded at admission.

Follow-up permission is admission authority. Notification style, unread counts, ranking, and dismissing a prompt are presentation preferences and do not decide the request. A disabled follow-up **MUST** fail without offering an automatic second charge against the pending request.

4 Fixed Terms and Canonical Time

4.1 What the sender authorizes

Before creating a request, the sender obtains signed terms containing the quote identifier; protocol, policy, and financial-program versions; scoped parties and service operator; relationship and request-history versions; C and S ; settlement unit; submission and decision durations; request eligibility time; quote issue and expiry times; and refund and forfeiture rules. The terms **MUST** include the

applicable corporate-share rate or an immutable reference to it. Any declaration-dependent pricing or eligibility **MUST** bind a canonical digest of the initial message’s authenticated declarations.

The base charge is

$$Q = C + S.$$

The service operator sets C , a bounded menu for S , and a default selection. A recipient may choose from that menu through an authenticated preference. The C-SQD defaults are specified in the deployment profile; the protocol does not embed a particular price or currency. The full amount and possible outcomes **MUST** be disclosed before authorization. The corporate share is an allocation from eligible forfeitures, not an additional sender fee. Quoted financial terms **MUST** remain immutable after creation.

Recipient preferences affect new quotes only. A preference change **MUST NOT** alter the amount on an existing request. The base request has no additional persistence reserve, and follow-ups add no financial component.

4.2 Time and receipt authority

Every state-changing command **MUST** have an authoritative receipt time and one canonical linearization position. Device clocks cannot determine financial timeliness. Authenticated recipient decisions **MUST** be durably received before the provider issues a signed receipt or claims they were received.

Creation materializes the submission deadline from canonical creation time. Submission materializes one immutable decision deadline T from canonical submission time. Arithmetic **MUST** be checked for overflow. A message’s separate validity deadline only gates that message’s admission; it cannot shorten or extend T , alter a submitted request’s outcome, or authorize early deletion of content still required for delivery.

A decision is timely when durably received at or before T . Expiry **MUST NOT** become final until the journal establishes that no applicable timely decision remains unprocessed. A bounded processing holdback may permit sequencing of already received decisions; it does not extend the recipient’s decision window.

5 Financial Obligations and External Execution

5.1 One request, one charge

The service **MUST** associate every charge operation with the authorized request and original payment reference. An uncertain provider result is not a failed charge: it **MUST** be reconciled under the original operation key before any replacement is dispatched. Multiple messages, uploads, devices, or payment notifications **MUST NOT** multiply the authorized economic effect.

The deployment **MUST** define the evidence required to admit a paid request. A local accounting entry alone is not evidence of external payment. Authorization, capture, settlement, dispute exposure, and refund confirmation are distinct facts. In particular, an authorization is not assumed to last throughout the recipient decision window. The C-SQD profile requires authenticated evidence that the charge satisfies its funding-finality policy before submission. A pending provider acknowledgment is insufficient. Later reversals remain possible and require explicit recovery; “finalized funding does not mean that dispute exposure has vanished.

5.2 The request outcome

The following table gives obligations created by a terminal outcome. “Refund” means recording an amount due against the original charge and scheduling its external execution, not claiming immediate receipt by the sender’s bank.

Outcome	Financial consequence
Accepted in time	Refund $C + S$ to the original sender.
Rejected in time	Retain C as processing revenue; record S as pending forfeiture.
Blocked in time while awaiting a decision	Apply the rejection allocation and prohibit future contact.
Expired without timely decision	Retain C ; refund S .
Cancelled before submission	Void an uncaptured authorization or refund the full captured amount.

Rejection **MUST NOT** establish a payment obligation to the rejecting recipient. Pending forfeitures become eligible for the C-SQD pool process only under the financial profile’s maturity rules. Refundable amounts **MUST NOT** be counted as distributable resources, regardless of where cash is held.

5.3 The ledger and the outbox

The ledger **MUST** use append-only double-entry accounting, checked integer amounts, and conservation within each settlement unit. It **MUST** distinguish processor clearing, unresolved request amounts, refund obligations, processing revenue, utility-service receipts, pending forfeitures, corporate allocations, restricted member funds, and finalized member payables. These are accounting categories, not general user spending balances. Refunds and member payables **MUST NOT** fund another request, transfer between users, or become on-demand wallet withdrawals.

A terminal request transition **MUST** atomically commit its relationship effect, request outcome, ledger obligations, scheduled-work changes, and external-effect intents. Workers execute those intents after commit. A payment or refund remains pending until authenticated provider evidence and reconciliation establish its completion. Failed or delayed payment **MUST NOT** erase the obligation.

Provider notifications **MUST** be authenticated, associated with the correct operation and amount, and deduplicated. Out-of-order evidence cannot regress a confirmed outcome. A late capture of a cancelled request creates or updates the full refund obligation; it cannot revive submission. Later external reversals require separately authorized compensating entries, not rewritten protocol history or automatic deductions from member entitlements.

5.4 Funding attempts and service coverage

The C-SQD host **MUST** bound new funding attempts under operator-controlled source verification and unresolved-attempt capacity. A failed or reversed funding source requires restriction and authenticated re-verification; bank-account replacement policy remains deferred. A queued reservation is not dispatch authority: the host **MUST** check current restrictions transactionally before granting a new dispatch. Creating a new request identifier **MUST NOT** bypass that restriction. Reconciliation of one uncertain operation preserves its identity and does not consume a new attempt.

Annual utility billing belongs to the host service, separately from the request lifecycle. Published terms fix a year-long service interval, price, and collection date well before that interval starts.

Failed or late retries **MUST** retain those terms and **MUST NOT** shift or extend the interval. Finalized funding authorizes coverage only within that interval, without retroactive service. Before issuing terms, creating or submitting a request, or admitting a follow-up, accepted message, or lane message, C-SQD checks the recipient's local coverage and the sender's coverage if the sender also has a local account. The remote sender's provider owns its own entitlement check.

Pending annual funding grants no new coverage. Expired coverage **MUST NOT** prevent recipient decisions, refund reconciliation, inspection of obligations, or payment of a finalized member allocation. It does not revoke a relationship or lane. An annual allocation is paid automatically as one lump sum to the associated verified bank account. Its rebate and excess amounts are reporting classifications, not separate processor operations. Distribution **MUST NOT** revoke purchased service, and next-year renewal or funding **MUST NOT** gate or reclassify it. Eligibility, annual allocation, and the distribution due date remain governed by the financial profile. The C-SQD profile assumes one configured payment-processing arrangement.

6 Commands Along the Request Lifecycle

Each command carries authenticated authority, expected versions, an idempotency key, and domain-separated signing context. Exact replay returns the recorded result; the same key with a different payload fails. Infrastructure retries **MUST** preserve the original intent and query an uncertain outcome before creating another operation.

6.1 Issue terms and create the request

`IssueRequestTerms` checks relationship state, current request, recipient policy, and principal-scoped history. A blocked pair is refused. Accepted ordinary communication requires no request charge. An existing nonterminal request is returned as such; only an eligible new request receives a charge quote.

`CreateRequest` validates the quote signature, expiry, unit, versions, initial-message binding, and sender authorization. Under the directed-pair uniqueness boundary it creates a `PreparingSubmission` request, fixes its terms, schedules submission timeout, and records one external payment intent. It creates no recipient solicitation and does not advance request history.

Stale terms, blocking, or standing acceptance prevent a new charge. So does an existing request in preparation or awaiting a decision. If an external operation was already in flight when such a state change occurred, its completion is reconciled and any resulting captured amount is refunded through the cancelled request rather than silently abandoned.

6.2 Submit the request to the recipient

`SubmitRequestToRecipient` validates current permission, request and quote versions, finalized funding evidence, submission and message-validity deadlines, authenticated declarations, and durable encrypted content. The lifecycle becomes `AwaitingRecipientDecision`. That transition **MUST** atomically record the single decision deadline, advance principal-scoped new-request history, replace submission timeout with expiry, and store the initial delivery intent and one logical solicitation effect.

If cancellation or timeout wins first, submission cannot revive the request. If acceptance occurs while it is preparing, cancel the request and return its full captured amount; the message may subsequently use accepted-sender admission. Blocking before submission cancels it in full and prevents delivery. No message becomes visible before its durable admission commits.

6.3 Admit a follow-up

`AdmitFollowup` identifies the exact request awaiting a decision and authenticates a new message under the same directed pair. At one canonical boundary it checks that the request is still awaiting a decision, the pair is not blocked, current recipient follow-up policy permits the message, and content and declarations are valid. It consumes permitted allowance once, associates the message, and records the delivery intent. It leaves charge, financial deadline, and new-request history unchanged.

Policy updates and follow-up admission **MUST** serialize on a version or equivalent guard. A request that closes first cannot accept a follow-up under stale authority. If standing acceptance now permits delivery, the sender may use that path with authenticated admission; a failed follow-up must not silently become a newly charged request.

6.4 Accept the relationship

`AcceptRelationship` is authorized by the recipient for the directed public-identity pair. It establishes accepted permission. If the current request is awaiting a decision and the decision was timely, the same transition marks it accepted, records exactly one full refund obligation, closes the solicitation, and cancels expiry. If the request is preparing, acceptance cancels it in full instead.

A late acceptance grants future permission but cannot reopen a logically expired or terminal request. Its expiry obligations must still be materialized. Accepting one public identity **MUST NOT** accept sibling identities or reset the private principal's new-request history.

6.5 Reject, block, unblock, and revoke

`RejectRelationship` declines the current unaccepted relationship request. A timely request awaiting a decision becomes rejected with the allocation defined above; a preparing request is cancelled in full. Rejection closes the solicitation and preserves history and applicable retry cooldown. It is invalid for an accepted relationship, which must first be revoked. A rejection received after the request deadline does not replace its expiry allocation.

`BlockRelationship` may operate from any relationship state. It sets the pair to blocked, invalidates capabilities, and closes any solicitation. A timely request awaiting a decision uses the rejection allocation; a preparing request is cancelled in full; a logically expired or terminal request keeps its existing outcome. An exact replay moves no further value.

While blocked, terms, creation, initial submission, follow-ups, and capability admission **MUST** be refused. `UnblockRelationship` returns the pair to rejected without accepting it, restoring a lane, erasing history, or reopening a request. `RevokeRelationship` changes accepted permission to revoked for future contact without altering prior request outcomes. New requests after either transition remain subject to current eligibility and recipient policy.

6.6 Expire or cancel

`ExpireRequest` acts only on a request awaiting a decision after the decision window and journal holdback rules permit finalization. It rechecks timely decisions, records expiry with processing revenue and collateral refund, closes the solicitation, and schedules the refund. It **MUST NOT** undo a later prospective acceptance. A stale job targeting a terminal request has no further effect.

`CancelRequestSubmission` may be authorized by the sender, a verified pre-submission failure, or submission timeout. It marks the request cancelled, voids or refunds its payment, cancels timeout, and creates neither a solicitation nor a history increment. Cancellation is valid only before submission. Closing a client or losing a network connection is not itself a cancellation command.

7 Ordering, Retry, and Abuse Continuity

Creation, submission, follow-up policy changes, and recipient decisions **MUST** serialize for the directed pair. Decisions carry an expected relationship version; the first valid committed decision changes it and stale conflicting decisions fail without partial effects. Submission and cancellation share the request version. No execution may both advance request history and cancel the same request before submission.

Timely receipt must survive queue delay. A worker cannot win an expiry race merely by running before a decision already durably received by the deadline. Conversely, a new late decision cannot retroactively change the request’s financial outcome. Subsequent processing of an older request must preserve current permission.

Request history is keyed by (P_i, I_j) through a scoped assertion. It contains a new-request level, eligibility time, policy version, and canonical event position. Initial submission advances it atomically; follow-ups and cancelled preparation do not. Acceptance bypasses that history for the accepted public identity without erasing it for aliases. Cross-identity request creation **MUST** observe the shared history boundary so concurrent aliases cannot all use stale initial terms.

The deployment defines cooldowns after expiry and rejection, recipient overrides, and explicit invitation rules. Any decay or forgiveness of history requires a versioned policy event. Blocking remains effective regardless of willingness to pay. These controls limit renewed requests; follow-up limits govern messages inside the current one.

8 Stable Errors and Recovery

Errors **MUST** distinguish a refused protocol operation from an unresolved external operation. In the following table, refusal creates no new charge or partial financial effect; an already captured cancelled request still requires its full refund.

Condition	Required behavior
AuthenticationFailed, InvalidSignature QuoteExpired, QuoteVersionStale	Reject without state effects. Refuse new creation; obtain fresh terms only if no request exists.
RelationshipAccepted RequestAlreadyExists	Use accepted-sender admission; do not charge. Return the scoped existing request or conflict; do not create another charge.
ContactBlocked, BackoffActive	Refuse contact or report eligible time without advancing history.
PaymentPending	Query/reconcile the original operation; do not dispatch a replacement.
PaymentFailed, AmountMismatch	Do not admit; reconcile and cancel/refund any captured amount under the original authorization.
SubmissionWindowClosed RequestClosed, FollowupNotAllowed DecisionWindowClosed	Cancel preparation and void/refund payment in full. Refuse the follow-up without a new charge. Preserve expiry economics; acceptance may grant prospective permission.
AlreadyTerminal, Duplicate	Return the recorded result for exact replay. A conflicting payload fails.
VersionConflict MessageValidityClosed	Refresh state; apply no partial outcome. Refuse this message; cancel a still-preparing initial request, but do not settle a submitted request because a follow-up is stale.

Condition	Required behavior
CapabilityScopeMismatch	Consume no allowance and charge nothing automatically.
UnsupportedCriticalExtension	Refuse admission dependent on the extension; unwind a still-preparing initial request.
PrivacyScopeViolation, LinkageErased	Refuse excess disclosure or reconstruction of erased linkage.
ContentKeyUnavailable	Return an endpoint recovery error; do not request provider decryption or change settlement.

9 Authenticated Envelopes and Scoped Capabilities

The native envelope contains a versioned opaque message identifier, scoped parties, creation and validity times, encrypted-content reference, declared purpose, accountable origin declaration, optional payload-schema identifier, and the request or permission proof needed for admission. Every admission-relevant field **MUST** be authenticated and bound to the encrypted content.

Purpose and origin use constrained, versioned values, including explicit unspecified and legacy values. Unknown noncritical schemas may remain opaque; unsupported critical extensions fail before admission. The envelope **MUST NOT** expose plaintext, legal identity evidence, unrelated identities, financial balances, or content-derived classifications unnecessary for the chosen path.

The reference deployment supports express lanes as recipient-signed scoped capabilities. Each grant **MUST** bind its subject and recipient, declared-purpose constraint, optional origin constraint, acceptable declaration authority, hard validity interval, enforceable count or rate, version, and revocation rules. An opaque handle identifies the pair; it does not supply authority without the grant. Admission compares authenticated declarations exactly under versioned matching rules. It does not verify the semantic truth of encrypted content.

For legacy senders, a DMARC pass under RFC 9989 is evidence of authorized use of the visible author domain, not proof of purpose, safety, company identity, or human authorship. The gateway **MUST** identify itself as declaration authority and use legacy/unspecified origin unless separate authenticated evidence supports more. The lane binds a recipient-selected canonical IDNA-ASCII domain scope, accepting the domain or its authorized subdomains under a consistent versioned mapping. A passing DKIM or SPF identifier cannot silently replace that scope.

The admission order is blocked refusal, accepted permission, valid capability, permitted follow-up on the identified request awaiting a decision, then eligible new-request creation with explicit sender authorization. If a request already exists but a follow-up is refused, the last branch is unavailable. Capability mismatch never silently converts a message into a charged request.

Capability-authorized delivery creates no request, forfeiture, solicitation, or new-request history increment. Grant, renewal, expiry, and revocation do not settle an existing request. Blocking invalidates capability authority; unblocking does not restore it. The lane's activity clock cannot extend a request deadline or establish distribution-program activity by itself.

10 Security and Privacy Requirements

Operational signing keys must be rotatable without changing the private principal. Recovery and revocation **MUST** prevent a compromised device from retaining accepted or capability authority. Neither recovery nor distribution verification authorizes provider access to native content keys.

10.1 Native Content Confidentiality

A conforming native client **MUST** encrypt message content before transmitting it to a provider. Only sender-authorized and recipient-authorized endpoints **MAY** hold the ordinary content-decryption keys. A provider, ledger operator, content store, or delivery worker **MUST NOT** require possession of those keys as a condition of native delivery.

The cryptographic binding between the encrypted content and its authenticated envelope **MUST** prevent substitution, truncation, reordering, and attachment replacement. The recipient **MUST** be able to authenticate the sending protocol identity and detect an unexpected operational-key change. Implementations **SHOULD** provide auditable key transparency or an equivalent mechanism that makes silent key substitution detectable.

Content storage **MUST** use ciphertext references rather than embedding content in relationship or ledger events. Server-side indexes **MUST NOT** contain plaintext subjects, bodies, attachments, or content-derived terms. Search over native content **MUST** occur at an authorized endpoint or through a construction that does not disclose the searched plaintext to the provider.

An SMTP gateway or other legacy bridge **MAY** necessarily encounter plaintext or provider-readable content. Such delivery **MUST** be identified as outside the native content-confidentiality guarantee. A provider **MUST NOT** represent ordinary TLS transport or provider-managed encryption at rest as native end-to-end encryption.

10.2 Pairwise Relationship Privacy

The logical relationship remains (I_i, I_j) , but components that do not route or display an address **MUST** represent it by an opaque pairwise handle H_{ij} . The handle **MUST** be generated randomly or by a secret-key construction with domain separation and at least 128 bits of effective unpredictability. It **MUST NOT** be a plain, unsalted, or otherwise publicly testable hash of the two protocol identities.

A handle is scoped to one directed identity pair and one service domain. The handle used for I_i contacting I_j **MUST NOT** reveal the handle used for the same sender contacting another recipient. The reverse relationship from I_j to I_i **MUST** use a distinct handle. Relationship, content, ledger, and observability systems **SHOULD** use separately scoped references so that one global identifier does not join all records without passing through a controlled translation boundary.

The private new-request relationship (P_i, I_j) similarly **MUST** be represented outside the identity service by a recipient-scoped opaque assertion. That assertion **MUST** allow the correct anti-persistence history to survive changes of sender address, key, or provider, while preventing unrelated recipients from comparing assertions to discover a common private principal.

A provider migration **MUST NOT** export a globally stable principal identifier. Where continuity across providers is supported, the prior identity authority **MUST** instead issue or authorize a migration proof scoped to the recipient and the new-request purpose. The receiving provider **MUST NOT** be able to use that proof to discover the principal's relationships with other recipients.

There **MUST NOT** be a public or ordinary administrative interface that enumerates the global relationship graph. A recipient **MAY** inspect its own visible relationships, and a sender **MAY** inspect its own attempts and receipts, but neither party receives another participant's unrelated edges. Pending, rejected, ignored, revoked, blocked, and accepted relationships receive the same graph-confidentiality protection.

An accepted relationship **MAY** persist as an opaque handle or recipient-issued capability together with current state, version, and revocation information. The system **MUST NOT** retain

the complete formation history merely because current permission remains active.

10.3 Compartmentalization and Minimal Disclosure

The implementation **MUST** separate identity evidence, relationship state, encrypted content, financial settlement, and operational telemetry into distinct access domains. This is a separation of authority as well as storage. Access to one domain **MUST NOT** automatically grant access to the keys or translation data of another.

Within ordinary operation, the identity service **MAY** know that a verified principal controls a public identity, but it need not receive message content. The relationship service **MAY** know H_{ij} and its state, but it need not receive legal identity evidence or plaintext content. The ledger **MAY** know opaque account, request, and settlement references together with amounts, but it need not receive a subject, content reference, public address pair, or private principal identifier. The content store **MAY** know a ciphertext object and a scoped retrieval reference, but it need not receive a request outcome or readable endpoint identities.

The delivery boundary necessarily learns enough to route to I_j and to present I_i to the recipient. It **MUST** disclose no additional principal evidence, unrelated identities, balance information, or relationship history. A recipient provider **MUST** verify the applicable request admission, follow-up authority, or capability without receiving unrelated financial activity. External providers **SHOULD** receive only the request charge, original-payment refund, or member payment references and amounts needed for execution. They do not need message content, follow-up events, or a correspondence graph.

Translation among scoped references is itself sensitive processing. Every translation **MUST** be authorized for a specific protocol purpose and recorded in an access audit that does not reproduce the translated graph. Bulk translation, graph export, and use of relationship data for advertising, sale, or unrelated profiling are outside the protocol purpose and **MUST NOT** be enabled by default.

10.4 Membership and Obligation Privacy

Distribution verification **MUST** use its own scoped member references, access roles, and retention policy. An eligibility assertion may establish one qualifying member without exposing legal evidence to delivery or the allocation engine. Activity evidence **MUST** be limited to facts needed by the published rule; message content and unrelated correspondents are not eligibility inputs. Unpaid refunds and finalized payables retain necessary payment/audit evidence under their own retention classes even after ordinary message linkage expires.

10.5 Metadata Leakage and Traffic Analysis

Pairwise handles do not conceal every observable fact. Timing, network addresses, message sizes, uncommon request amounts, and synchronized journal events can permit correlation without a direct identifier. Implementations **MUST** document which of these signals each actor can observe and **MUST** retain them only while needed for routing, security, settlement, or a disclosed obligation.

Where it does not compromise correctness, an implementation **SHOULD** reduce correlation through coarse timestamps, payment batching, content padding, separation of network and account logs, and short retention of delivery telemetry. These protections **MUST NOT** alter quoted amounts, obscure a settlement from its owner, or make request finality ambiguous.

10.6 Record Lifecycle and Retention

Every stored field **MUST** belong to a documented retention class stating its purpose, access domain, controlling key, operational lifetime, dispute lifetime, any separately applicable required-retention period, and deletion method. A vague interest in future analytics is not a protocol purpose for indefinite retention.

While a request is pending, the protocol **MAY** retain the scoped linkage required to authenticate admission, prevent replay, apply new-request policy, receive a decision, and settle the request. Once the request is terminal and its defined dispute period ends, the implementation **MUST** remove fields and indexes that are not needed for continuing permission, accounting, fraud handling, or an applicable retention obligation. A durable accounting record **SHOULD** contain only an opaque settlement reference, amount, settlement unit, terminal state, canonical time, and the minimum ownership evidence required for audit.

Continuing acceptance does not justify continuing retention of individual contact attempts. After acceptance, the live relationship **MAY** retain H_{ij} , current state, version, capability or key references, and revocation material. Formation messages, quote details, delivery telemetry, and superseded relationship events **MUST** follow their own shorter retention schedules unless independently needed.

Records retained solely for a legal, accounting, fraud, or dispute obligation **MUST** be segregated from online delivery and product analytics. They **MUST** use separate encryption keys, narrowly authorized access, an explicit expiration condition, and an audit trail. A hold on one record or account **MUST NOT** silently extend retention for unrelated users or relationships.

Deletion **MUST** cover primary storage, replicas, caches, search indexes, telemetry, derived datasets, and backups according to a documented propagation schedule. If immediate removal from an immutable backup is impracticable, the backup **MUST** remain inaccessible to ordinary processing, expire within a fixed period, and lose access to deleted record keys no later than that period.

10.7 Cryptographic Erasure and Consent-Locked Archives

Encryption is not erasure while the operator retains both a usable key and enough linkage to identify the parties. When a retention period ends, the implementation **SHOULD** delete the unnecessary record and destroy every operator-held key that could recover its identifying linkage. A claim of cryptographic erasure is valid only when replicas, wrapped key copies, recovery escrow, and backups can no longer restore that linkage.

The protocol **MAY** support a consent-locked historical archive in which reconstruction requires independent sender and recipient key shares. Such an archive **MUST** be separate from live permission and settlement state. Refusal or loss of a share **MUST NOT** prevent the protocol from delivering under current permission, applying revocation, enforcing backoff, or completing a pending settlement.

If an implementation offers two-party reconstruction, it **MUST** state precisely which fields it covers and whether any separately retained compliance record can identify the transaction without both shares. It **MUST NOT** promise two-party-only reconstruction for information that the operator knowingly retains under an independent decryption key. Required records that cannot be consent-locked **MUST** be minimized and isolated under the preceding retention rules.

No deletion or threshold scheme can erase a message or receipt independently kept by a sender, recipient, endpoint backup, or external provider. The protocol's guarantee concerns records controlled by conforming providers and **MUST** describe that boundary accurately.

10.8 Operational Logs and Analytics

Logs **MUST NOT** contain plaintext content, private identity evidence, raw key material, full account balances, or a reusable global relationship identifier. Debugging and tracing identifiers **MUST** be scoped and rotated so that an observability system does not become an accidental relationship graph.

Analytics **SHOULD** operate on aggregates produced after the shortest useful window. A dataset is not anonymous merely because names and addresses have been removed; if scoped handles, rare amounts, precise timestamps, or auxiliary data permit practical reidentification, it remains protected relationship data and **MUST** follow the corresponding access and retention rules.

11 Incentives and Policy Authority

The operator receives processing revenue on rejection or expiry and, under the C-SQD profile, a disclosed share of eligible forfeitures. Recipients do not earn a personal rejection bounty, but member duplication, fabricated activity, and operator manipulation remain relevant risks. Providers **MUST** make acceptance available, issue signed decision receipts, and preserve timely decisions rather than manufacture expiry. Pricing and financial-program policies **MUST** include review, dispute, correction, and audit procedures.

Protocol policy fixes request economics and deterministic transitions. Admission policy evaluates authenticated envelopes, permission, recipient follow-up settings, rate/history state, funded local service coverage, and authorized safety evidence. Presentation policy runs for the recipient and controls ranking, notifications, digests, and local content processing. Neither latter policy may rewrite charge terms or terminal outcomes. A learned or probabilistic recommendation is not an authenticated relationship command. Admission decisions **MUST** reduce to versioned, auditable rules over authorized inputs.

Payment does not bypass content or safety controls, and those controls do not authorize decryption of native content. Providers may use recipient reports, endpoint evaluation, metadata, and deliberately disclosed evidence within their authorized purposes. Sender-facing errors should explain actionable constraints without exposing unrelated recipient rules or enabling enumeration.

12 Interoperability and Migration

Versioned exchanges **MUST** cover identities, scoped continuity assertions, request terms and identifiers, payment/admission evidence, message envelopes, capabilities, recipient decisions, outcome receipts, and financial obligations. Equivalent canonical inputs must yield equivalent request outcomes. Signatures and idempotency domains **MUST** prevent reuse across deployments and versions.

Federation must preserve scoped references, native encryption, request uniqueness, history continuity, and financial finality. It cannot assume that old direct recipient-collateral clearing applies to the company pool. A separate agreement must assign charge/refund responsibility, pool governance, membership uniqueness, clearing, default losses, and payment obligations before real federated value is enabled. SMTP downgrade must be visible before transmission and in the envelope.

13 Required Invariants and Conformance Stories

Conformance **MUST** preserve the following together:

1. One nonterminal request per directed pair, one authorized charge per request, and no additional financial effect from follow-ups or retries.
2. One request deadline fixed at submission; timely-decision priority and no retroactive reopening of a terminal or logically expired request.
3. Full return before submission; no ordinary cancellation after submission.
4. Atomic permission, request outcome, ledger obligations, and outbox intent, with external confirmation represented separately.
5. Exact outcome allocation, integer value conservation, and no recipient entitlement created by rejection.
6. No distributable use of refundable amounts, no repeated corporate assessment of carryforward, and no disappearance of unpaid finalized obligations.
7. Directed public-identity permission with private principal history continuity, including concurrent aliases, without public identity linkage.
8. Blocking overrides every admission authority; unblocking does not accept, restore capabilities, erase history, or reopen money.
9. Authenticated declarations, current follow-up authorization, replay-safe allowance consumption, and message-validity isolation.
10. Native content confidentiality, pairwise scoping, compartmentalization, bounded retention, and honest erasure across replicas and backups.

A reference suite should begin with one request and three permitted messages: acceptance creates one refund obligation and future permission. It should then exercise disabled follow-ups, concurrent creation, alias rotation, cancellation versus submission, a late capture after cancellation, timely decisions delayed in queues, late acceptance, duplicate and out-of-order provider evidence, and failed refund recovery. Profile tests must additionally exercise maturity exclusions, once-only corporate allocation, annual eligibility, integer rounding, zero eligible members, carryforward, and failed member payments. They must also cover advance collection for fixed annual service intervals, unchanged terms across failed or late retries, and equal allocations independent of renewal. For $U = \$120$, allocations of \$80, \$120, and \$170 must each produce one bank payment, with reporting values $R_i = \min(D_i, U)$ and $X_i = \max(0, D_i - U)$. Tests must cover automatic payment to the associated verified bank account, one completion lifecycle with lost-response recovery, and payment after closure. Future renewal funding must not delay or reclassify a distribution. The earlier full service charge remains separate, and distribution does not revoke coverage. Pending funding must grant neither request submission nor new service coverage.

The implementation sequence follows the same story: model request transitions and obligations; establish the transactional journal and outbox; add authenticated payment evidence and reconciliation; integrate encrypted delivery and follow-up controls; then validate the C-SQD financial program with simulated external effects. Live rails and federation follow only after their deployment policies are explicit. The desktop pilot may use a nonredeemable simulator without claiming that it has validated live payment operations.